PROYECTO 1: DFSha, SISTEMA DE ARCHIVOS DISTRIBUIDO CON ALTA DISPONIBILIDAD

Informe técnico final · Fase II (implementación grupal)

Miguel Moreno Rojas · Abel Santiago Garcia Ortiz

1. Introducción

En un sistema distribuido suele pensarse que la disponibilidad se compra con complejidad: más réplicas, más protocolos y más código de coordinación. Sin embargo, buena parte de esa complejidad ya está resuelta en herramientas maduras. La pregunta del equipo dejó de ser cómo programar el consenso y pasó a ser qué parte del problema vale la pena construir y qué parte conviene delegar.

Esa idea guió la Fase II. Partimos de dos diseños individuales que coincidían en lo esencial y diferían en un punto: cómo mantener vivo el plano de control. El de Miguel proponía un primario y un respaldo coordinados con un journal propio; el de Abel delegaba los metadatos a un clúster etcd que ya implementa Raft. Elegimos combinar los dos: la estructura simple del primero y la base de consenso del segundo.

El resultado es un sistema de archivos en el que cada archivo se divide en bloques de 4 MB, cada bloque se guarda cifrado en tres nodos de datos y los metadatos viven en etcd. El cliente no conoce dónde están los datos: lo pregunta en cada operación. El prototipo quedó desplegado en tres máquinas EC2 de AWS Academy, y sus seis pruebas funcionales pasaron 38 de 38 verificaciones, mientras que la de rendimiento midió hasta 90,5 MB/s de lectura agregada.

Se implementaron RF1 (árbol de directorios por usuario), RF2 (put y get de archivos completos) y RF3 (open, read, write, lock y close), junto con replicación, consistencia, tolerancia a fallos, autenticación, autorización y cifrado. El código está en el repositorio **mig006/Proyecto_DFSha**, donde también se ven los commits de cada integrante.

2. De dos propuestas a una arquitectura

Al comparar las propuestas se notó algo revelador: coincidían en casi todo lo que define la corrección del sistema, como los bloques inmutables, R = 3 con W = 2, los locks con lease y la semántica close-to-open. La diferencia estaba en el costo de implementarlo. Por eso la matriz se leyó con una pregunta: ¿qué opción cumple la misma garantía con menos código propio?

2.1 Matriz comparativa

Tabla 1

Comparación de las propuestas individuales y decisión del equipo

Aspecto	Propuesta de Miguel	Propuesta de Abel	Decisión del equipo
Plano de control	Primario y respaldo con journal, épocas y testigos	Instancias meta sin estado; metadatos en etcd (Raft)	Abel: el consenso ya está resuelto
Metadatos	En memoria, respaldados por un journal	En etcd, con transacciones con comparación	Abel
Locks	Lease propio por rango de bloques	Lease de etcd por rango, con fencing token	Mezcla: lease de etcd, un lock por archivo
Tamaño de bloque	4 MB	8 MiB	Miguel: más bloques, reparto más visible
Bloques	Inmutables, copy-on-write	Inmutables, copy-on-write	Igual en ambas
Réplicas	R = 3, W = 2, en cadena	R = 3, W = 2, en cadena	R = 3, W = 2; el cliente escribe en paralelo
Primitivas de RF3	open, read, write, lock, close	Modos r, w, a, r+, flush, mv	Miguel: menos primitivas, igual garantía
Protocolos	REST y gRPC	REST y gRPC	Todo REST sobre HTTPS (FastAPI)
Seguridad	JWT, TLS 1.3, mTLS, AES-256-GCM	JWT Ed25519, Argon2id, mTLS, AES-GCM con HKDF	JWT, scrypt, tokens HMAC, HTTPS, mTLS en etcd, AES-256-GCM
Descubrimiento	Nombre DNS del control	DNS con las direcciones de meta	Lista de meta en DFS_META
Reparación de copias	La hace el control primario	Coordinador elegido con lease	Una meta líder elegida con lease en etcd

2.2 Registro de decisiones

Cada decisión del equipo se registró con su motivo y con el costo que se aceptó a cambio, porque una decisión sin costo declarado suele esconder un problema que aparece después.

Tabla 2

Registro de decisiones arquitectónicas

#	Decisión	Por qué	Costo aceptado
D1	Metadatos en 3 miembros de etcd	Raft tolera 1 caída de 3 sin código de consenso propio	Sin quórum no hay operaciones de metadatos
D2	Dos meta sin estado	Si una cae, el cliente usa la otra	Ninguno relevante
D3	Bloques de 4 MB inmutables	Las réplicas no divergen y nunca se ve un bloque a medias	Cambiar 1 byte reescribe 4 MB
D4	R = 3, W = 2	Se escribe con un nodo caído y la tercera copia se repone sola	Ventana breve con 2 copias

#	Decisión	Por qué	Costo aceptado
D5	El cliente sube a las réplicas en paralelo	Más simple que la cadena; si un nodo falla prueba el siguiente	Usa más ancho de banda del cliente
D6	Lock por archivo con lease de 30 s	Se libera solo si el cliente muere; el commit lo verifica (fencing)	Escritores de bloques distintos se serializan
D7	Commit en una transacción de etcd	Exige lock vigente, versión base y W copias vivas	Ninguno
D8	Semántica de sesión	Garantía simple y comprobable	Cambios ajenos visibles al reabrir
D9	Todo REST/HTTPS	Un solo framework, fácil de probar	Menos eficiente que gRPC para binarios
D10	Heartbeat cada 3 s; caído a los 15 s	Sin cola de mensajes	Unos 15 s para detectar una caída
D11	GC de bloques huérfanos tras 10 min	Limpia sin romper a lectores con versiones viejas abiertas	Espacio ocupado unos minutos

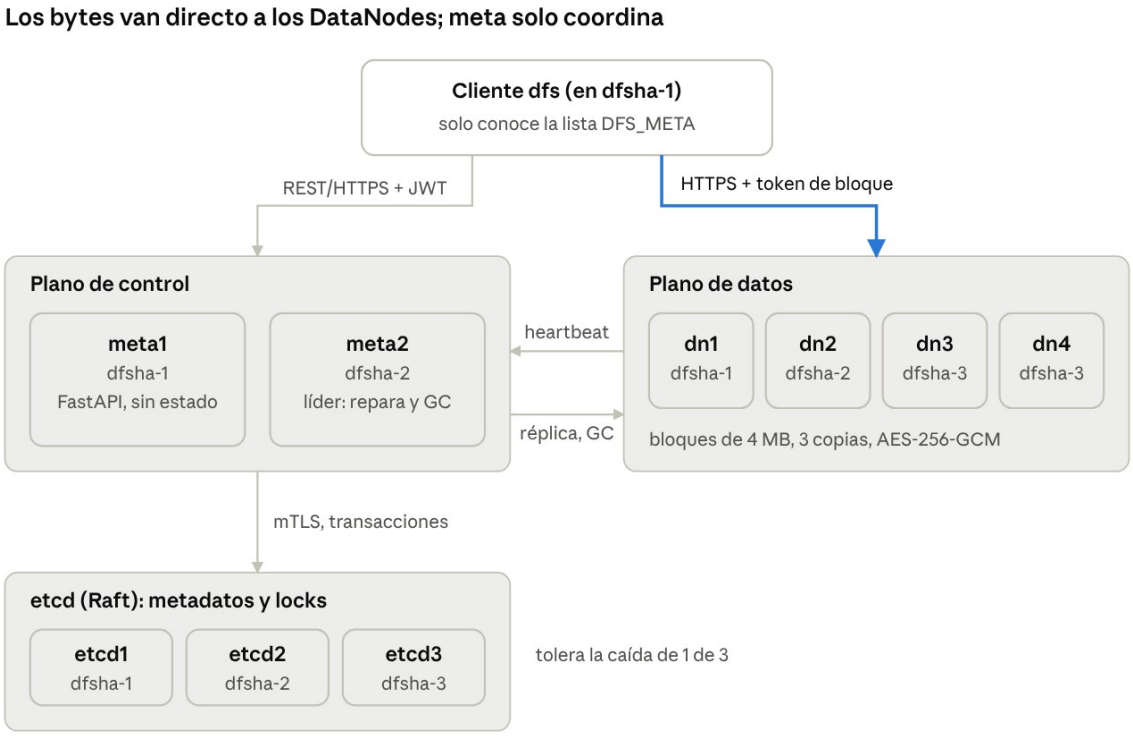
3. Arquitectura final

El sistema separa un plano de control, que decide dónde va cada bloque, de un plano de datos, que guarda y transfiere los bytes. Los datos nunca pasan por el control, así que el ancho de banda total crece con los DataNodes (RNF1, RNF5).

Esta separación funciona como una biblioteca con catálogo: el catálogo (etcd) dice en qué estantería está cada libro, pero quien lo busca va directo a la estantería. Si el catálogo estuviera en el camino de cada lectura, se volvería el cuello de botella.

Figura 1

Arquitectura desplegada: componentes, máquinas y comunicaciones



El cliente pide a meta el mapa de bloques y luego habla directo con los DataNodes (Figura 1). Las instancias de meta guardan todo en etcd, y la que es líder ordena las copias que faltan.

Tabla 3

Componentes del sistema

Componente	Instancias	Responsabilidad
Cliente dfs (CLI y librería Python)	Uno por usuario	Divide y reconstruye archivos, transfiere bloques en paralelo, verifica hashes, renueva locks
meta (FastAPI, sin estado)	2	Autentica, resuelve rutas, asigna bloques, concede locks y confirma versiones; la líder repara copias y limpia bloques
etcd	3 (Raft)	Guarda todos los metadatos con lecturas y escrituras linealizables, leases y transacciones
DataNode (FastAPI)	4	Guarda bloques cifrados, valida tokens de bloque, envía heartbeats y copia bloques cuando la líder lo ordena

Todo el estado del sistema cabe en cinco familias de claves de etcd, y por eso cualquier instancia de meta puede atender cualquier petición.

Tabla 4
Claves en etcd

Clave	Contenido	Vida
/users/<u>	Hash scrypt de la contraseña	Permanente
/fs/<u><ruta>	Directorio, o archivo con versión, tamaño, SHA-256 y lista de bloques con sus réplicas	Permanente
/dn/<id>	DataNode vivo: URL, espacio libre, número de bloques	Lease de 15 s
/locks/<u><ruta>	Lock exclusivo: lock_id, lease, usuario	Lease de 30 s
/leader	Instancia de meta que repara y limpia	Lease de 10 s

4. APIs y semántica de acceso

Toda la comunicación es REST sobre HTTPS. El cliente habla con meta usando un JWT y con los DataNodes usando tokens de bloque firmados, válidos durante 10 minutos.

4.1 API de meta

Tabla 5
Endpoints de meta (prefijo /api/v1)

Método y ruta	Uso	Primitiva
POST /auth/login	Devuelve el JWT (1 h)	login
GET /fs/ls, GET /fs/stat	Listar y consultar atributos	ls, cd, stat
POST /fs/mkdir, DELETE /fs/rmdir, DELETE /fs/rm	Crear y borrar	mkdir, rmdir, rm
POST /locks, PUT /locks/{id}, DELETE /locks/{id}	Pedir, renovar y soltar el lock	lock, unlock
POST /blocks/alloc	Ids de bloque, nodos destino y tokens de escritura	put, write
GET /files/locate	Mapa del archivo: bloques, copias vivas y tokens de lectura	get, open, read
POST /files/commit	Publica la versión nueva (atómico e idempotente)	put, close
GET /admin/cluster	Nodos vivos y líder	—

4.2 API del DataNode

Tabla 6
Endpoints del DataNode

Método y ruta	Uso
PUT /blocks/{id}	Guarda un bloque: verifica el SHA-256, cifra, escribe <id>.part y lo renombra
GET /blocks/{id}	Devuelve un bloque descifrado
POST /internal/replicate	Copia un bloque desde otra réplica (orden de la líder)

Método y ruta	Uso
GET /internal/blocks, DELETE /internal/blocks/{id}	Inventario y borrado para el GC

Los errores siguen una forma única, {"detail": {"error": CÓDIGO, "message": …}}. Los principales son 401 TOKEN_INVALID, 404 NOT_FOUND, 400 INVALID_OFFSET, 409 VERSION_CONFLICT, STALE_LOCK o DIR_NOT_EMPTY, 422 INSUFFICIENT_REPLICAS, 423 LOCKED y 503 NOT_ENOUGH_NODES.

4.3 Semántica de read() y write()

Tabla 7
Semántica declarada de RF3

Aspecto	Decisión declarada
Creación	Sí, con put u open(ruta, "w")
Reemplazo completo	Sí: put sobre un archivo existente crea la versión siguiente
Anexado	Sí: write(offset = tamaño actual)
Modificación parcial	Sí, en cualquier offset hasta el tamaño actual; se reescriben los bloques afectados
Huecos	No: offset > tamaño responde 400 INVALID_OFFSET
read(offset, n)	Lee la versión fijada al abrir; descarga en paralelo los bloques que abarca
Visibilidad	De sesión (close-to-open): los cambios se ven cuando close() los confirma
Concurrencia	Lectores sin límite y sin lock; un escritor por archivo a la vez

La garantía se formuló para que pudiera comprobarse. Una escritura está confirmada cuando commit responde 200 con la versión *v*. Desde ese momento se cumple lo siguiente:

1. cada bloque nuevo está en al menos 2 DataNodes vivos, con su hash verificado;
2. todo open() posterior, contra cualquier meta, ve *v* o una versión más nueva;
3. un lector ve todos los cambios de un commit o ninguno;
4. un descriptor abierto antes sigue leyendo su versión completa;
5. un escritor cuyo lock venció no puede publicar (409 STALE_LOCK).

5. Algoritmos

Toda la corrección del sistema descansa en una regla: un archivo solo cambia de versión mediante una única transacción de etcd. Lo demás, como subir bloques, reintentar o limpiar, puede fallar a mitad sin dejar nada a medias a la vista.

5.1 Escritura

Una escritura, sea un put o una sesión open(w), write y close, sigue seis pasos:

1. El cliente pide el lock del archivo (POST /locks). Si otro lo tiene, recibe 423 y reintenta con espera creciente.
2. Pide ids de bloque (POST /blocks/alloc). meta devuelve, por bloque, todos los DataNodes vivos ordenados por espacio libre y rota el nodo inicial para repartir la carga (RNF4).
3. Sube cada bloque a los tres primeros nodos en paralelo. Si uno falla, prueba con el siguiente de la lista, y exige al menos W = 2 confirmaciones.
4. Cada DataNode verifica el SHA-256, cifra el bloque, lo escribe en <id>.part, hace fsync y lo renombra a <id> antes de confirmar.
5. El cliente llama a POST /files/commit. meta comprueba en una sola transacción que el lock siga vigente, que el archivo esté en la versión base y que cada bloque nuevo tenga W copias en nodos vivos; solo entonces publica la versión siguiente.
6. El cliente suelta el lock. Mientras escribe, un hilo renueva el lease cada 10 s.

En una modificación parcial solo se reescriben los bloques que tocan los bytes cambiados. Los demás se conservan con el mismo id y las mismas réplicas.

5.2 Lectura

open pide el mapa del archivo (GET /files/locate) y lo fija. read(offset, n) calcula qué bloques abarca (índice = offset / 4 MB), los descarga en paralelo de cualquier réplica viva y verifica cada hash. Si una réplica falla o devuelve datos corruptos, usa la siguiente. get hace lo mismo con el archivo entero y compara además el SHA-256 completo.

5.3 Locks y fencing

Un lock es una clave en etcd atada a un lease de 30 s. Se concede con una transacción que solo escribe si la clave no existe, así que dos peticiones simultáneas nunca obtienen ambas el lock. Si el cliente muere, el lease vence y el lock desaparece solo.

El riesgo está en un cliente que se pausa, pierde su lock sin saberlo y despierta con datos que publicar. El commit lo impide, porque la misma transacción que publica compara el valor exacto del lock; si ya no es el suyo, responde STALE_LOCK.

5.4 Vivacidad, reparación y limpieza

Cada DataNode avisa a cualquier instancia de meta cada 3 s. meta renueva la clave /dn/<id> con un lease de 15 s; sin aviso, la clave vence y el nodo deja de recibir bloques. Por otra parte, cada meta intenta crear /leader con un lease de 10 s, y la que lo logra hace las tareas de fondo. Si cae, otra la reemplaza.

La líder recorre los archivos y, si un bloque tiene menos de 3 réplicas vivas, ordena a otro DataNode copiarlo desde una réplica sana y actualiza el mapa con una transacción. Además, cada 30 s borra los bloques que ningún archivo referencia y que tienen más de 10 minutos. Ese margen protege a los lectores que aún tienen abierta una versión anterior.

6. Seguridad

La seguridad se pensó en capas: ninguna sola protege todo, pero cada una cierra una puerta distinta. Las amenazas consideradas son un atacante en la red privada que escucha el tráfico, un usuario que intenta leer archivos ajenos, un token filtrado y un disco sustraído.

Tabla 8

Amenazas, mecanismos y evidencia

Amenaza	Mecanismo	Evidencia
Suplantar a un usuario	Contraseñas con scrypt y sal; JWT HS256 de 1 h	401 con contraseña mala, sin token o con token falso (prueba 6)
Leer archivos ajenos	Cada ruta se resuelve dentro de /fs/<usuario>; .. no escapa del espacio propio	404 al intentar leer los archivos de otro usuario (prueba 6)
Ir directo a un DataNode	Token de bloque HMAC-SHA256: un bloque, una operación, 10 minutos	401 sin token, con token alterado, de otro bloque o de lectura usado para escribir
Escuchar el tráfico	HTTPS en meta y DataNodes; mTLS entre meta y etcd y entre miembros de etcd; CA propia	HTTP plano rechazado y HTTPS aceptado
Robar un disco	AES-256-GCM por bloque, nonce aleatorio, id del bloque como dato autenticado	El texto no aparece en disco; el bloque son bytes ilegibles
Abusar del plano interno	Token interno compartido en heartbeats, replicación y GC	403 sin el token

En la red, el Security Group dfsha-sg solo abre SSH (22) a una IP. Los puertos de etcd (2379-2380), meta (8001) y DataNodes (9001-9002) solo aceptan tráfico de máquinas del mismo grupo. Los secretos y certificados viven en deploy/.env y deploy/certs, que el .gitignore excluye del repositorio.

7. Despliegue

El sistema corre en tres EC2 t3.small con Ubuntu 24.04, en la misma subred de us-east-1a, y cada componente es un contenedor Docker. Antes de ir a AWS se validó el mismo código en un clúster local de 9 contenedores en el portátil.

Figura 2

Clúster local con Docker Desktop: 3 etcd, 2 meta y 4 DataNodes

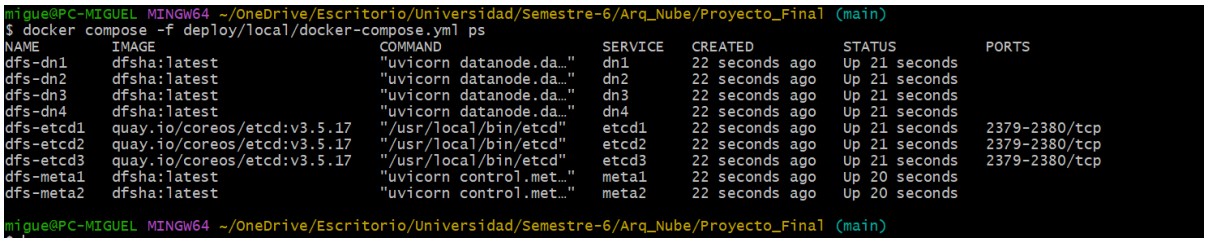
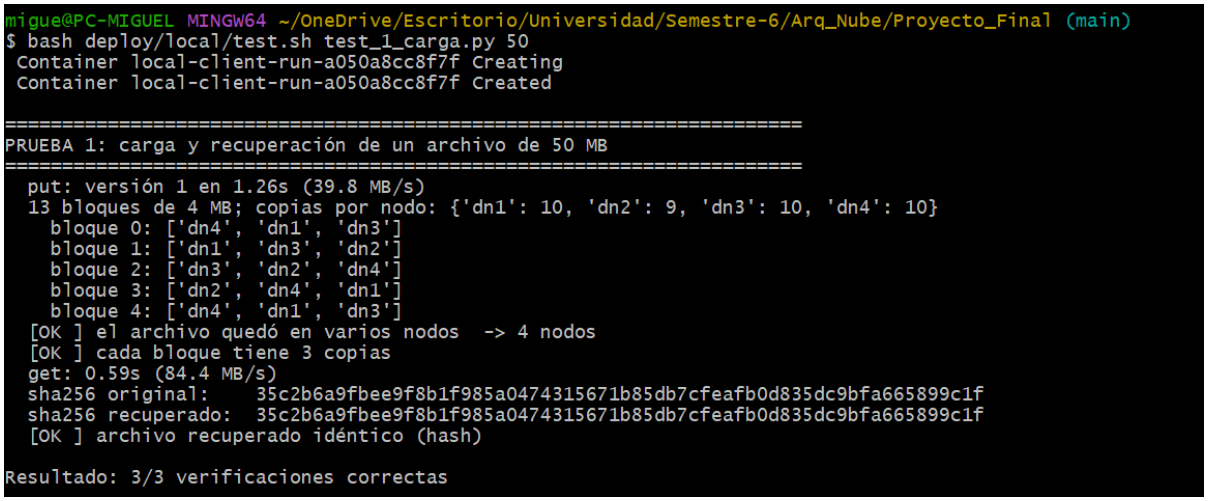


Figura 3

Prueba 1 en local: 50 MB en 13 bloques y hash idéntico



7.1 Infraestructura en AWS Academy

Tabla 9

Máquinas del despliegue

Máquina	IP privada	Contenedores
dfsha-1	172.31.16.187	dfs-etcd1, dfs-meta1, dfs-dn1 y el cliente
dfsha-2	172.31.16.245	dfs-etcd2, dfs-meta2, dfs-dn2
dfsha-3	172.31.30.194	dfs-etcd3, dfs-dn3, dfs-dn4 (puertos 9001 y 9002)

Los contenedores usan la red del host, así que cada servicio escucha en la IP privada de su máquina. Los certificados se generaron con esas tres IPs en el campo SAN y el mismo .env se copió a las tres. Cada máquina levanta solo su perfil con bash deploy/aws/up.sh host1, host2 o host3.

Figura 4

Las tres instancias en ejecución con sus IPs privadas, el grupo dfsha-sg y las métricas de red

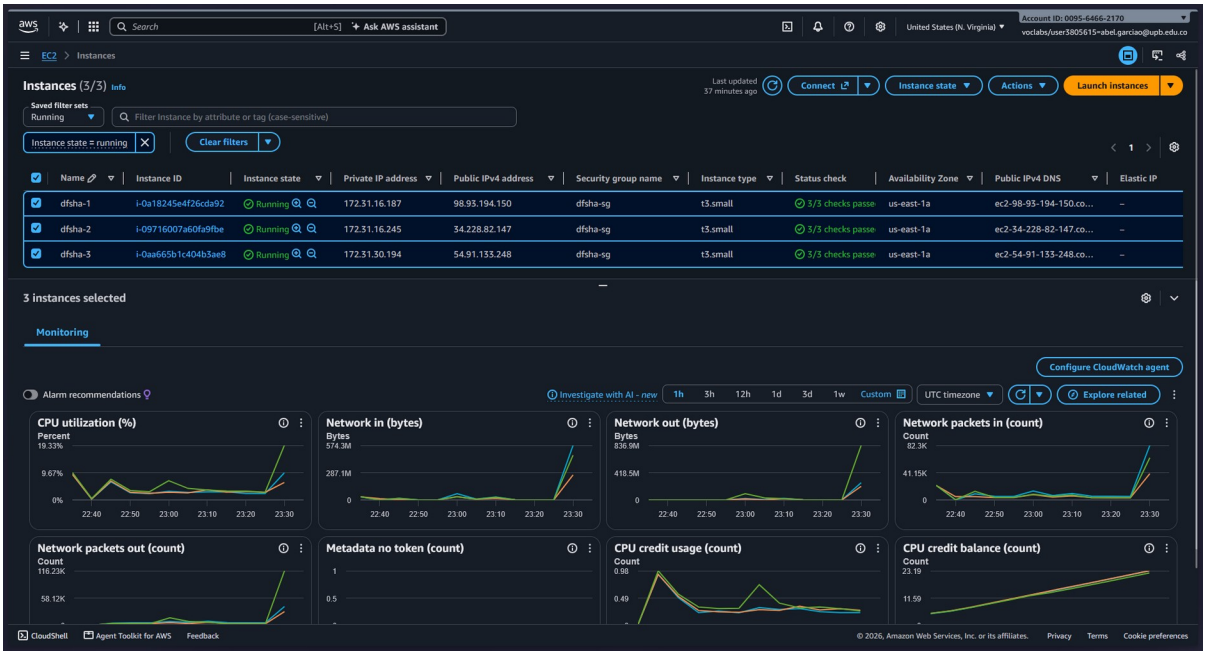


Figura 5

Reglas de entrada del Security Group dfsha-sg

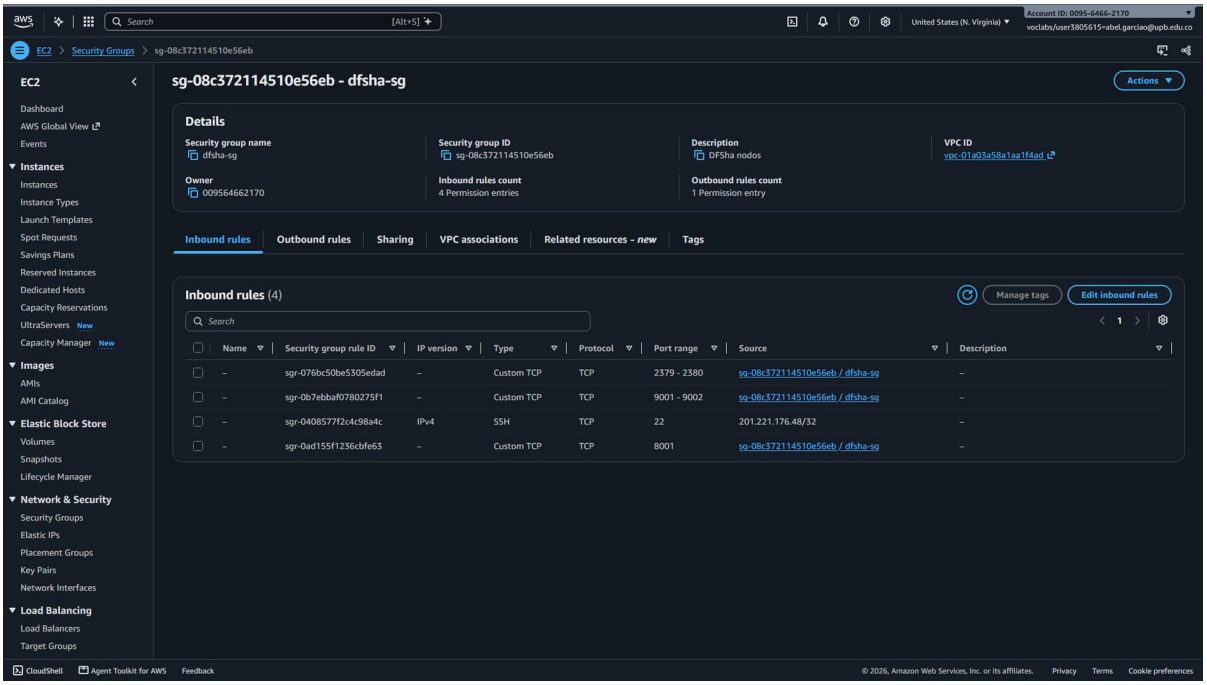


Figura 6

Contenedores en dfsha-1

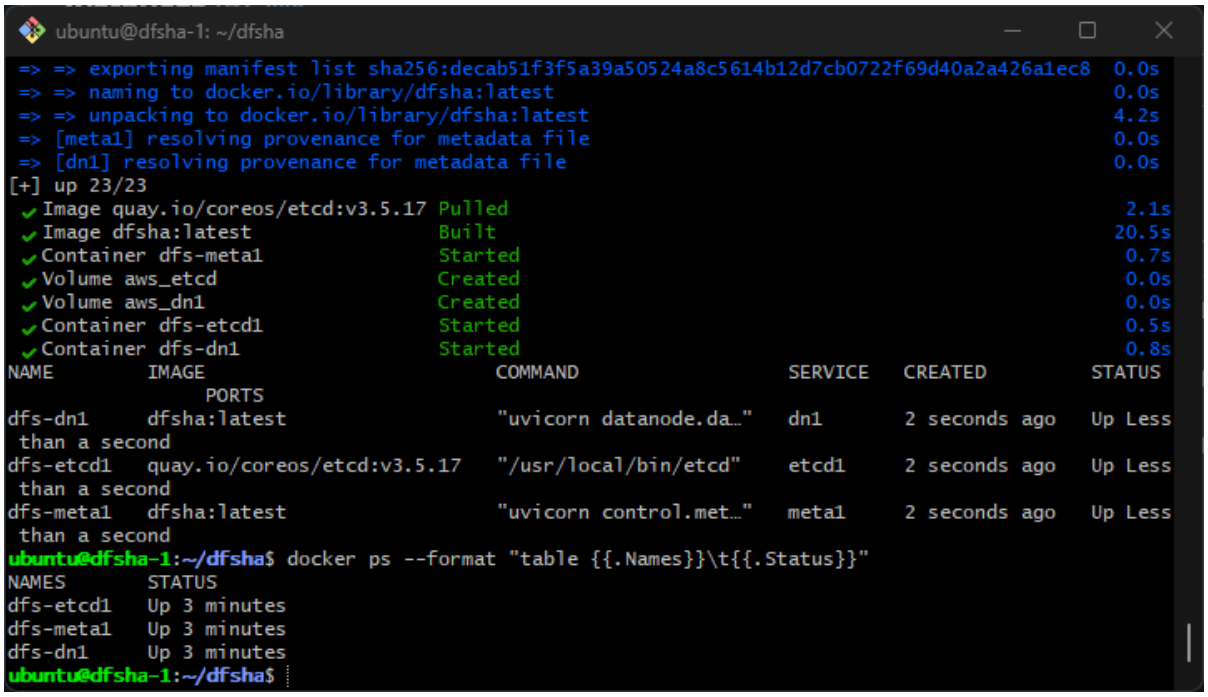


Figura 7

Contenedores en dfsha-2

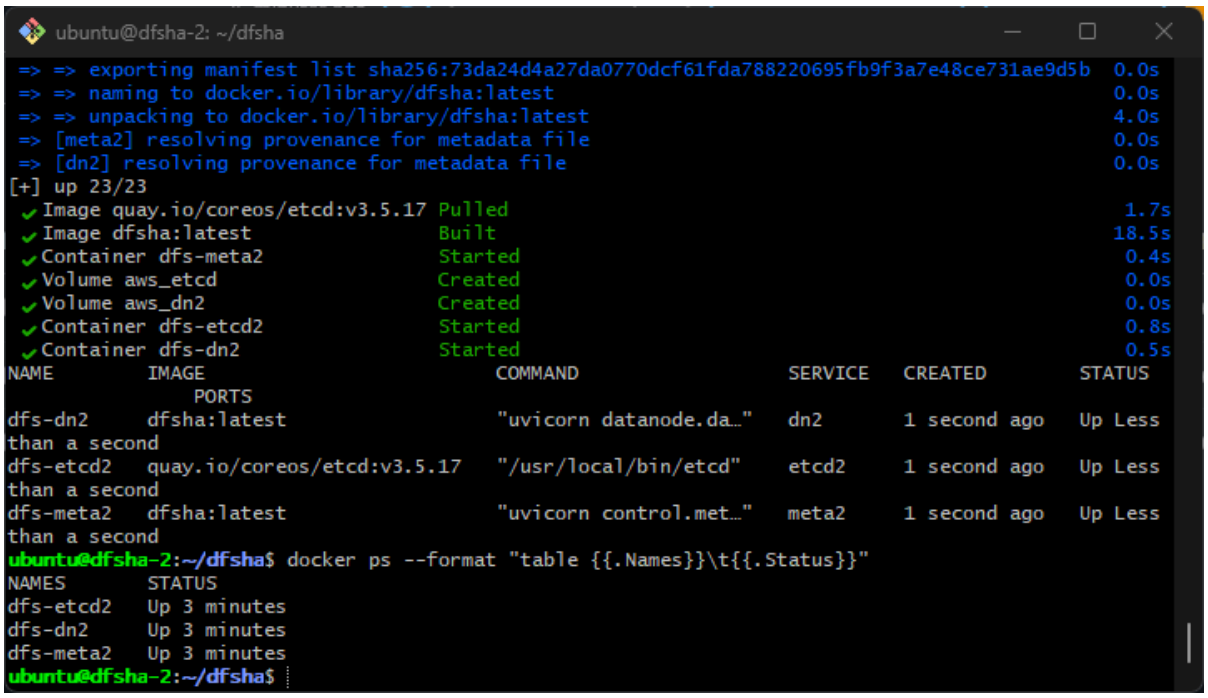
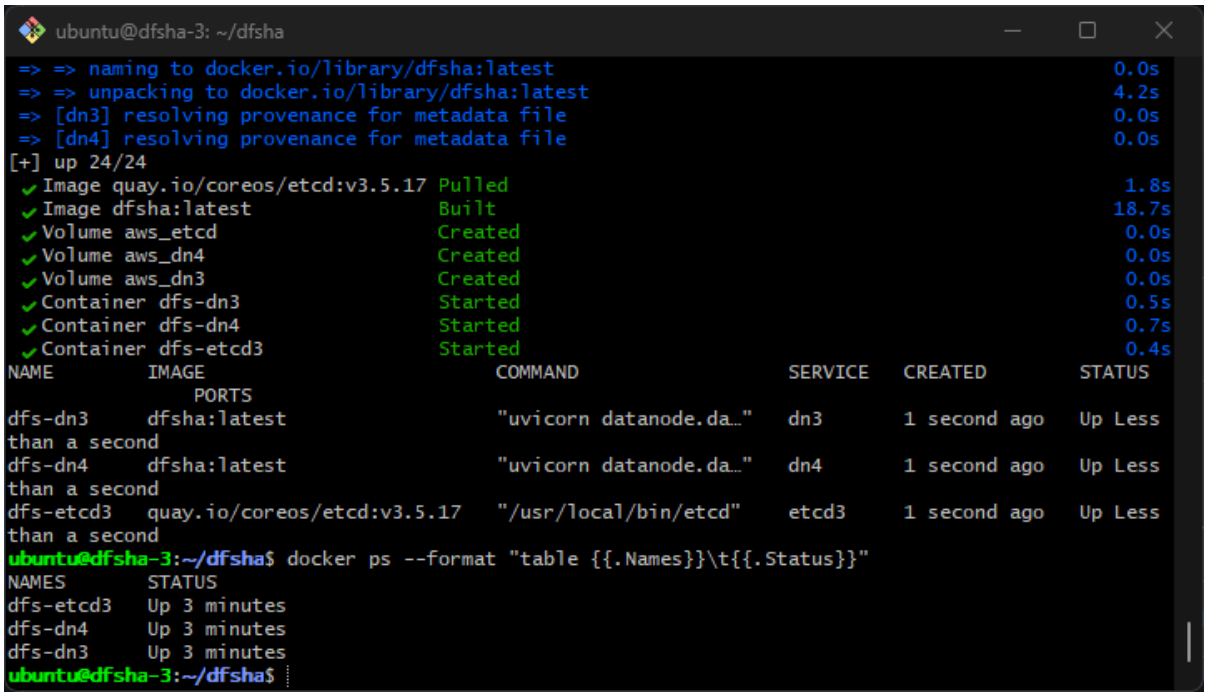


Figura 8

Contenedores en dfsha-3



7.2 Verificación del clúster

El comando cluster, ejecutado desde dfsha-1, mostró a meta2 como líder y los cuatro DataNodes registrados con la URL de su propia máquina (Figura 9). Los heartbeats llegaban por las dos instancias de meta, lo que confirma que ambas atienden peticiones. El estado de etcd mostró los tres miembros en la versión 3.5.17, con el mismo término de Raft y el mismo índice aplicado (7016); el líder de Raft era el miembro de dfsha-3 (Figura 10).

Figura 9

Salida de cluster: líder meta2 y cuatro DataNodes con sus IPs privadas

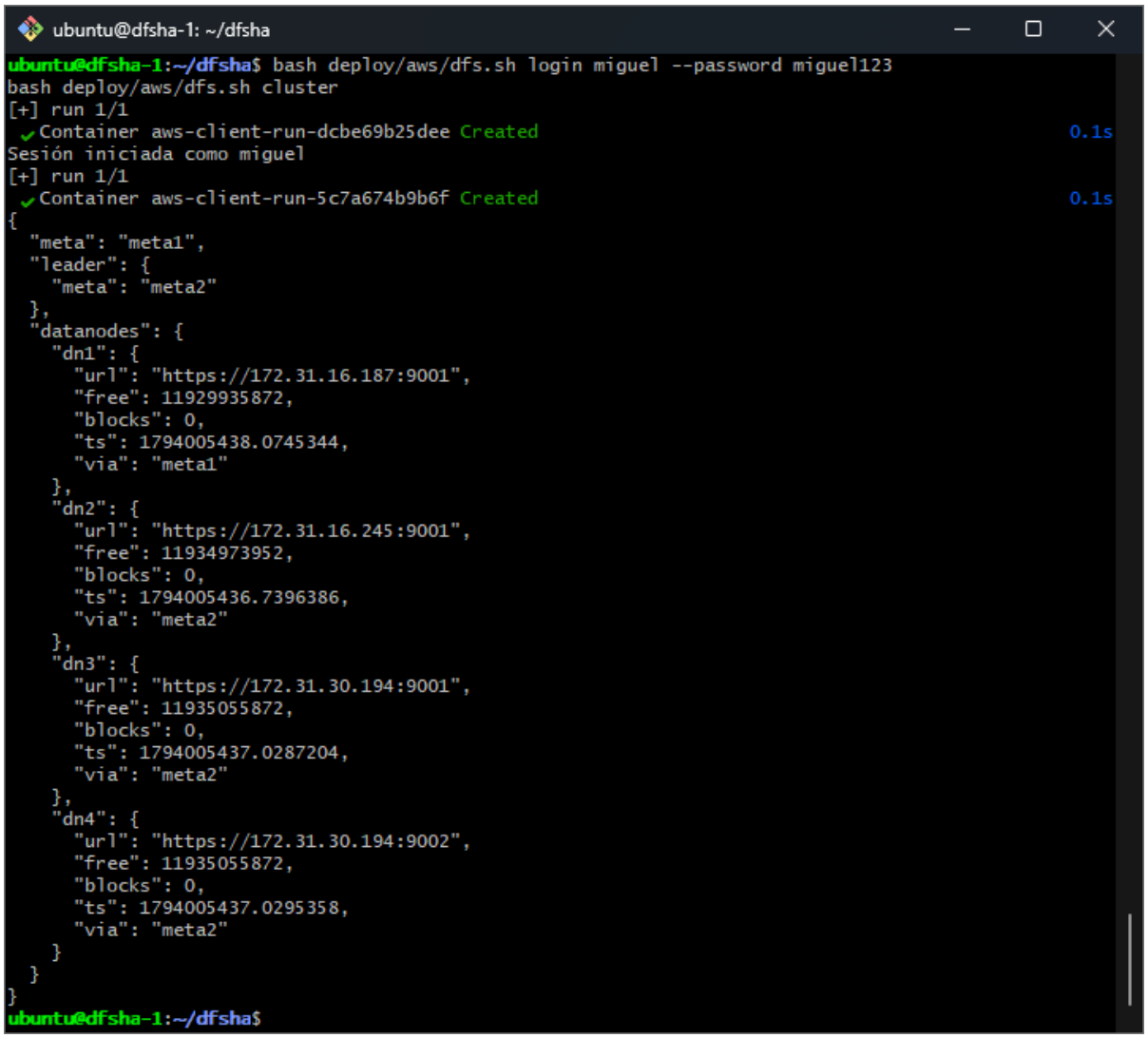
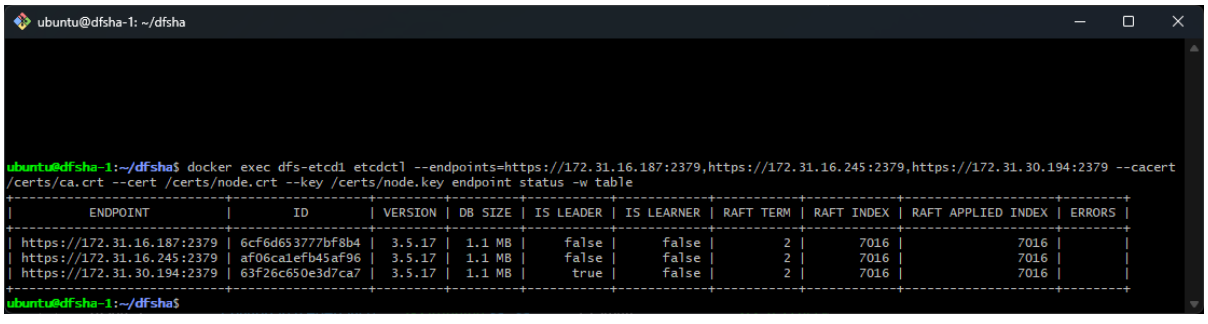


Figura 10

Estado del clúster etcd: tres miembros sincronizados

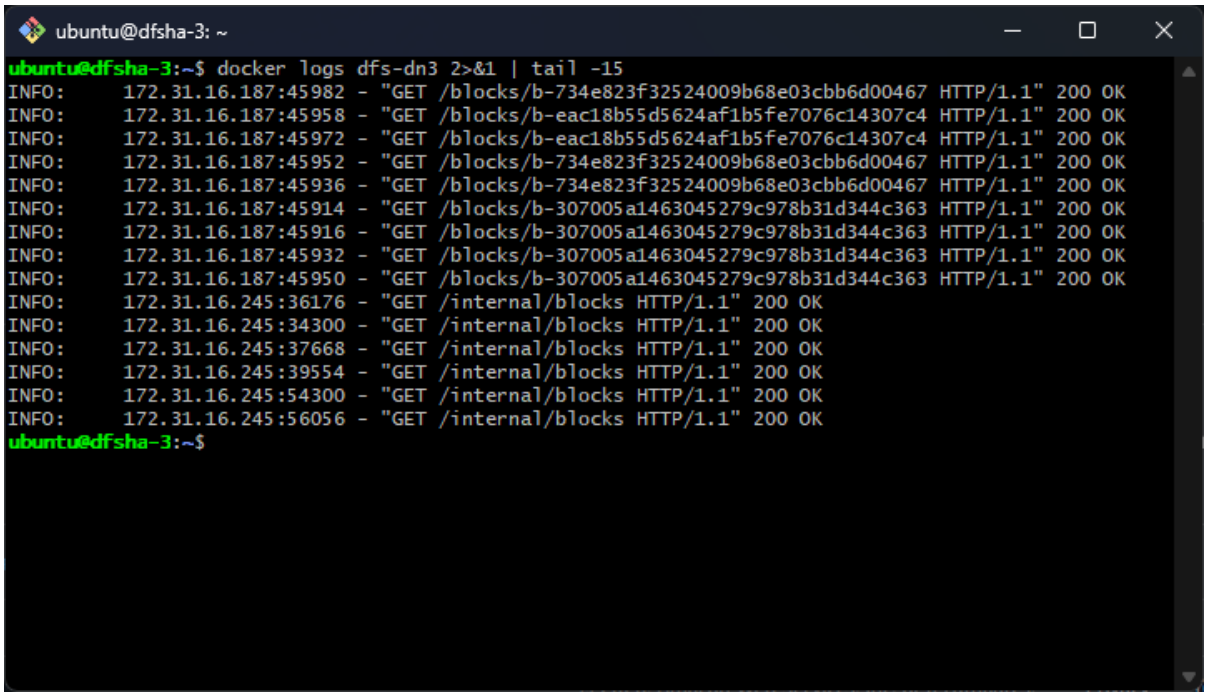


La comunicación entre máquinas se ve en los registros de dfs-dn3: peticiones GET /blocks/... desde 172.31.16.187, el cliente en dfsha-1, y GET /internal/blocks desde

172.31.16.245, meta2 haciendo el GC como líder (Figura 11). Las métricas de EC2 de la Figura 4 muestran el pico de red de las tres instancias durante las pruebas.

Figura 11

Registros de dfs-dn3 con tráfico desde las otras dos máquinas



8. Pruebas y resultados

Las seis pruebas funcionales pasaron todas sus verificaciones en AWS. Cada una es un script de la carpeta tests que imprime lo que hace y termina con un conteo de verificaciones, así que cualquiera puede repetirla con bash deploy/aws/test.sh y el nombre de la prueba.

Tabla 10

Resumen de las pruebas en AWS

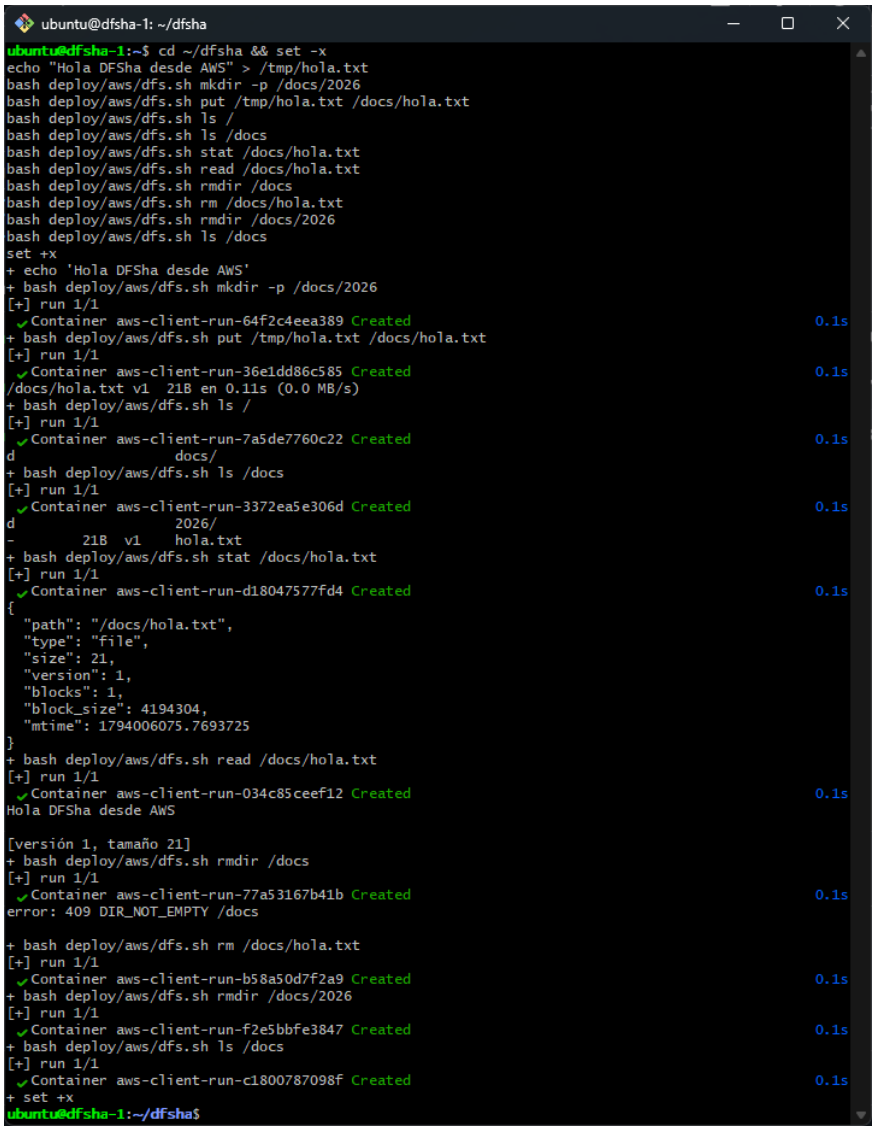
Prueba	Qué se demostró	Resultado
1. Carga y recuperación	200 MB en 50 bloques repartidos entre 4 nodos; SHA-256 idéntico	3/3
2. Acceso RF3	read, write parcial, anexado, huecos rechazados, lock exclusivo, visibilidad de sesión	7/7
3. Concurrencia	El segundo escritor esperó 4,32 s; sin pérdida ni mezcla; lock vencido rechazado	4/4
4. Replicación y fallos	Lectura con dn3 caído en 1,64 s; copias repuestas solas; sistema activo sin meta1 ni etcd1	5/5
5. Consistencia	Versión nueva con réplica desconectada; lector antiguo intacto; commit interrumpido sin efecto	7/7
6. Seguridad	Autenticación, autorización, tokens de bloque y canal cifrado	12/12
7. Rendimiento	Latencia y throughput en tres condiciones (sección 9)	Medida

8.1 Operaciones de RF1

Desde dfsha-1 se crearon directorios anidados, se subió un archivo, se listó, se consultó su stat (versión 1, un bloque de 4 194 304 bytes) y se leyó. Borrar un directorio con contenido falló como estaba declarado (409 DIR_NOT_EMPTY), y tras borrar el archivo, rmdir funcionó.

Figura 12

Secuencia de RF1: mkdir, put, ls, stat, read, rmdir rechazado, rm y rmdir



8.2 Prueba 1: carga y recuperación

Un archivo aleatorio de 200 MB se subió en 3,35 s (59,6 MB/s) y se descargó en 2,32 s (86,3 MB/s). Quedó en 50 bloques con 3 copias cada uno, repartidas casi por igual: 37, 37, 38 y 38 copias en dn1 a dn4. El SHA-256 del original y el del recuperado coinciden (2c344655…

a6e6fe1). El mapa de bloques muestra además la rotación del nodo inicial: el bloque 0 va a dn3, dn4 y dn2; el 1, a dn4, dn2 y dn1.

Figura 13

Prueba 1 en AWS con un archivo de 200 MB

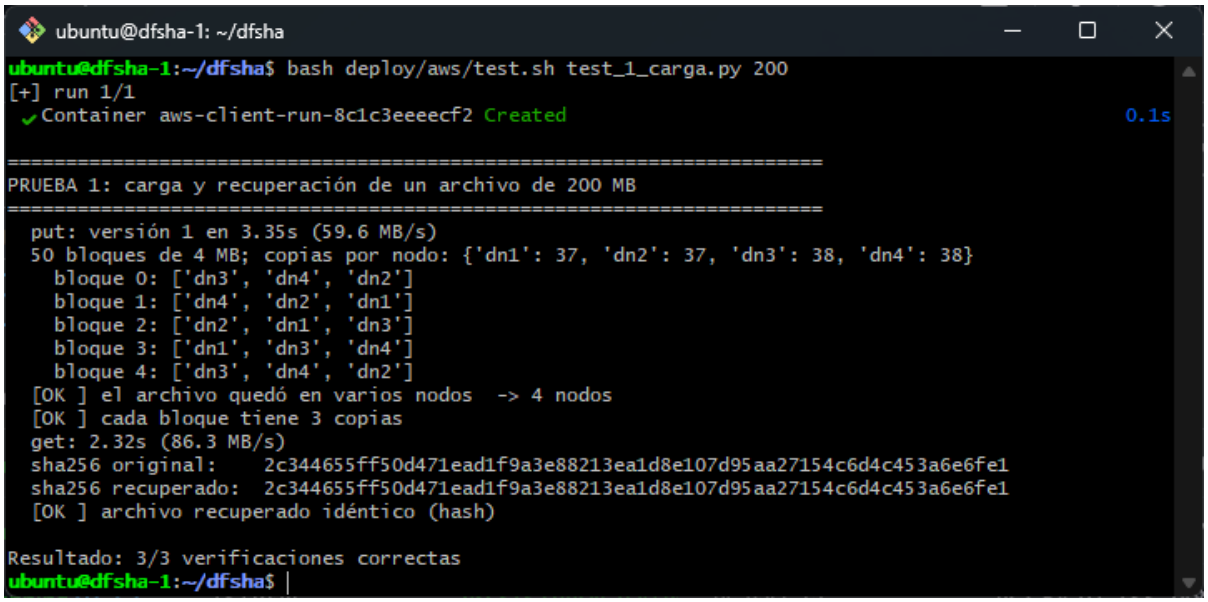
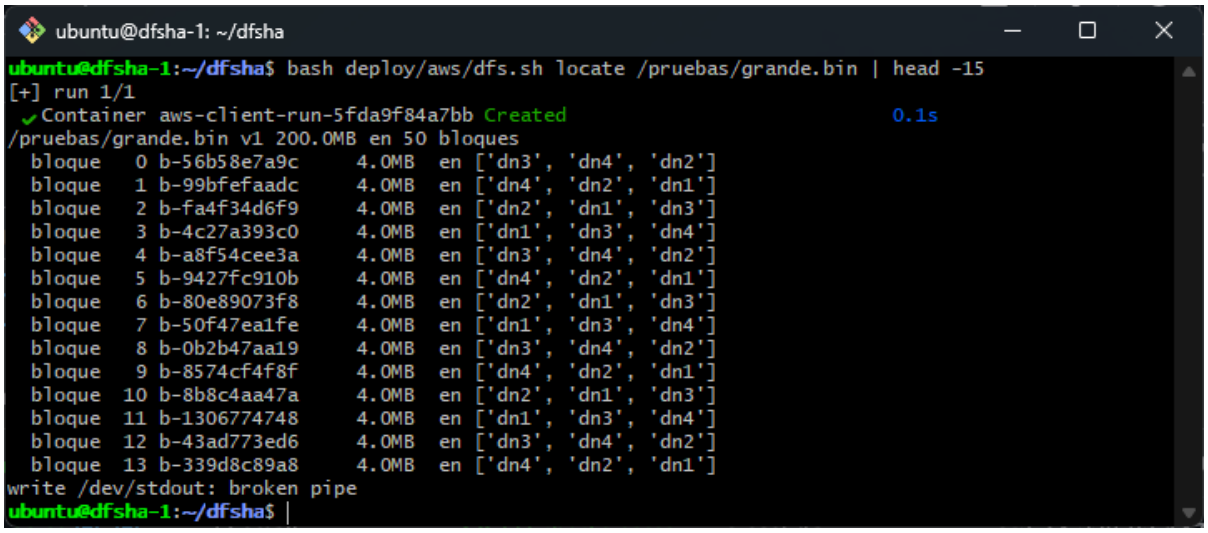


Figura 14

Mapa de bloques de /pruebas/grande.bin



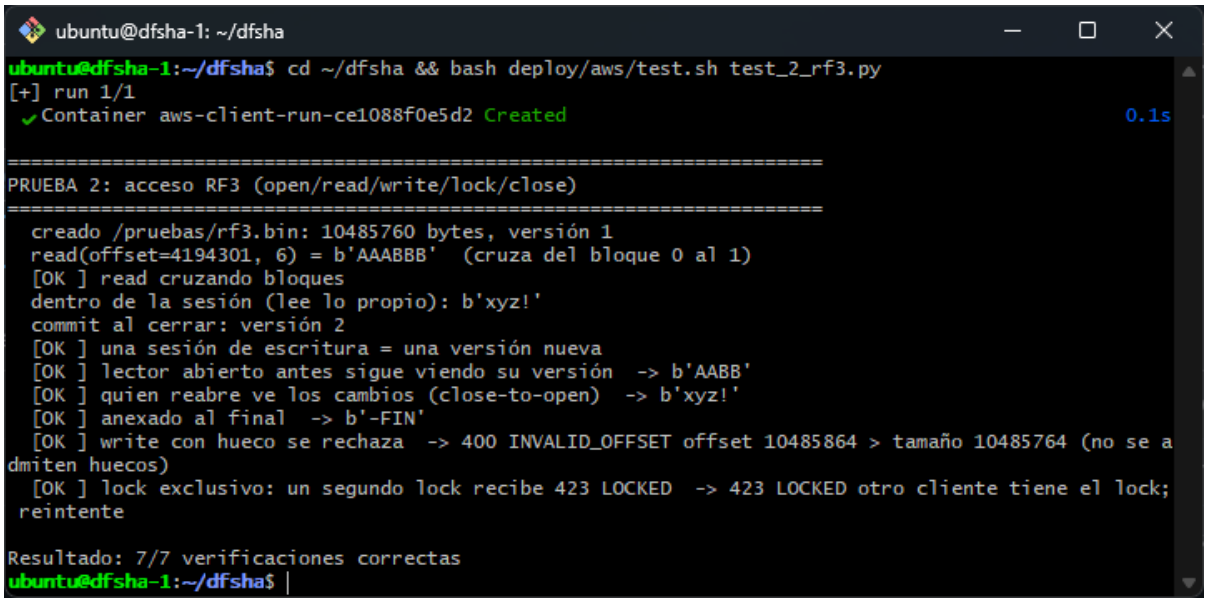
8.3 Prueba 2: acceso RF3

Sobre un archivo de 10 MB formado por bloques de 'A', 'B' y 'C', un read de 6 bytes que cruza del bloque 0 al 1 devolvió AAABBB. Una sesión de escritura modificó bytes en el límite entre bloques, anexó -FIN y publicó una sola versión nueva. Un lector abierto antes siguió

viendo AABB, mientras que uno que reabrió vio xyz!. Escribir más allá del final devolvió 400 INVALID_OFFSET, y un segundo lock, 423 LOCKED.

Figura 15

Prueba 2: semántica de RF3



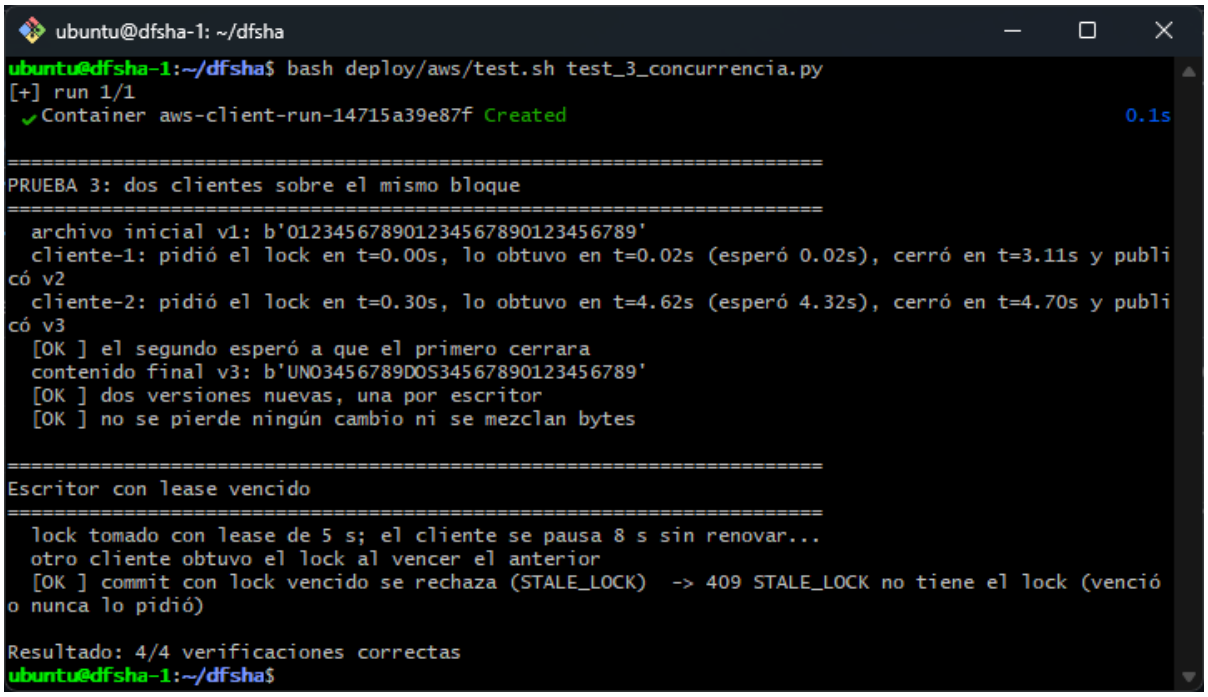
8.4 Prueba 3: concurrencia

Dos procesos abrieron el mismo archivo para escribir en el mismo bloque. El primero obtuvo el lock en 0,02 s, lo mantuvo 3 s y publicó la versión 2. El segundo lo pidió a los 0,30 s y lo obtuvo a los 4,62 s, es decir, esperó 4,32 s; de ese tiempo, unos 1,5 s se deben a la espera exponencial entre reintentos y no al lock. El contenido final (UNO3456789DOS3456789…) conserva los dos cambios sin mezclar bytes.

En la segunda parte, un cliente tomó un lock de 5 s y se pausó 8 s. Otro cliente obtuvo el lock al vencer el primero, y cuando el pausado intentó publicar, recibió 409 STALE_LOCK.

Figura 16

Prueba 3: escrituras serializadas y fencing



8.5 Prueba 4: replicación y fallos

Se subió un archivo de 40 MB (10 bloques, 30 copias) y se apagó dfs-dn3 con docker stop en dfsha-3. La lectura inmediata salió idéntica en 1,64 s usando las otras copias. Tras 16 s, meta marcó a dn3 como caído, y una escritura nueva quedó con 3 copias sin tocar dn3. Doce segundos después, la líder meta2 había repuesto las copias: el reparto pasó de 7, 7, 8 y 8 copias a 10, 10 y 10 en dn1, dn2 y dn4. Su registro muestra 49 copias repuestas en total, que incluyen los bloques de dn3 de los archivos de pruebas anteriores.

Luego se apagaron a la vez dfs-meta1 y dfs-etcd1. ls, get y put siguieron funcionando en 1,00 s, porque el cliente pasó a meta2 y etcd conservó el quórum con 2 de 3 miembros.

Figura 17

dn3 detenido en dfsha-3

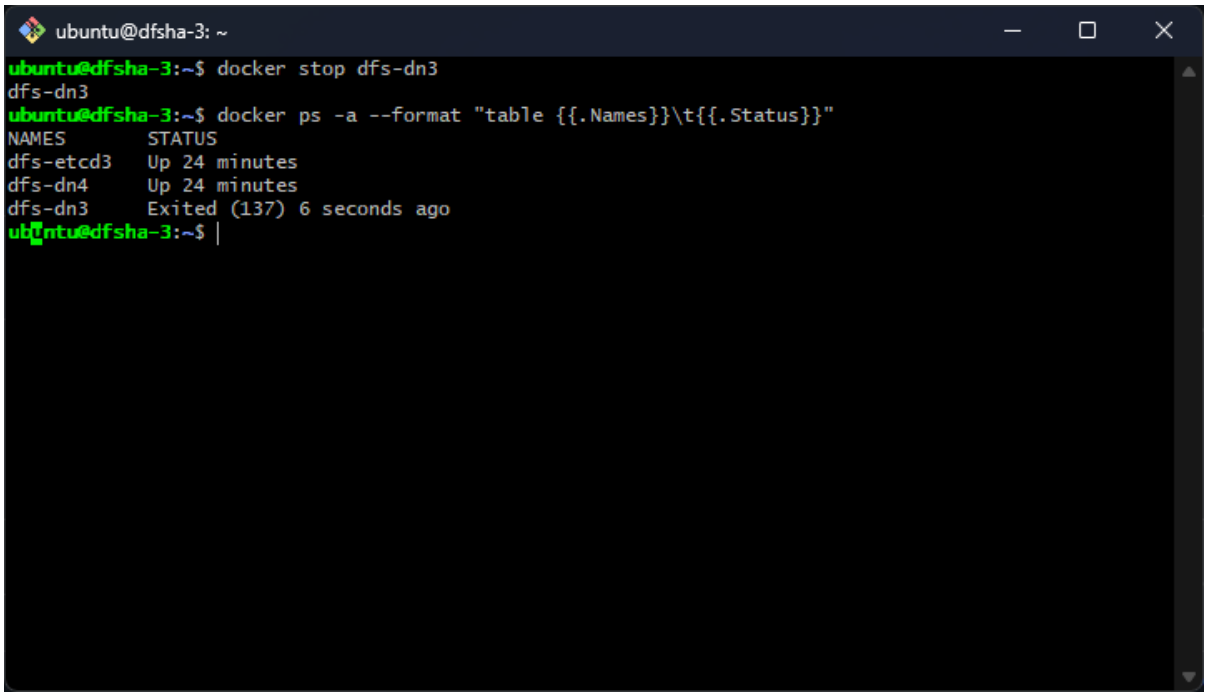


Figura 18

Prueba 4 completa

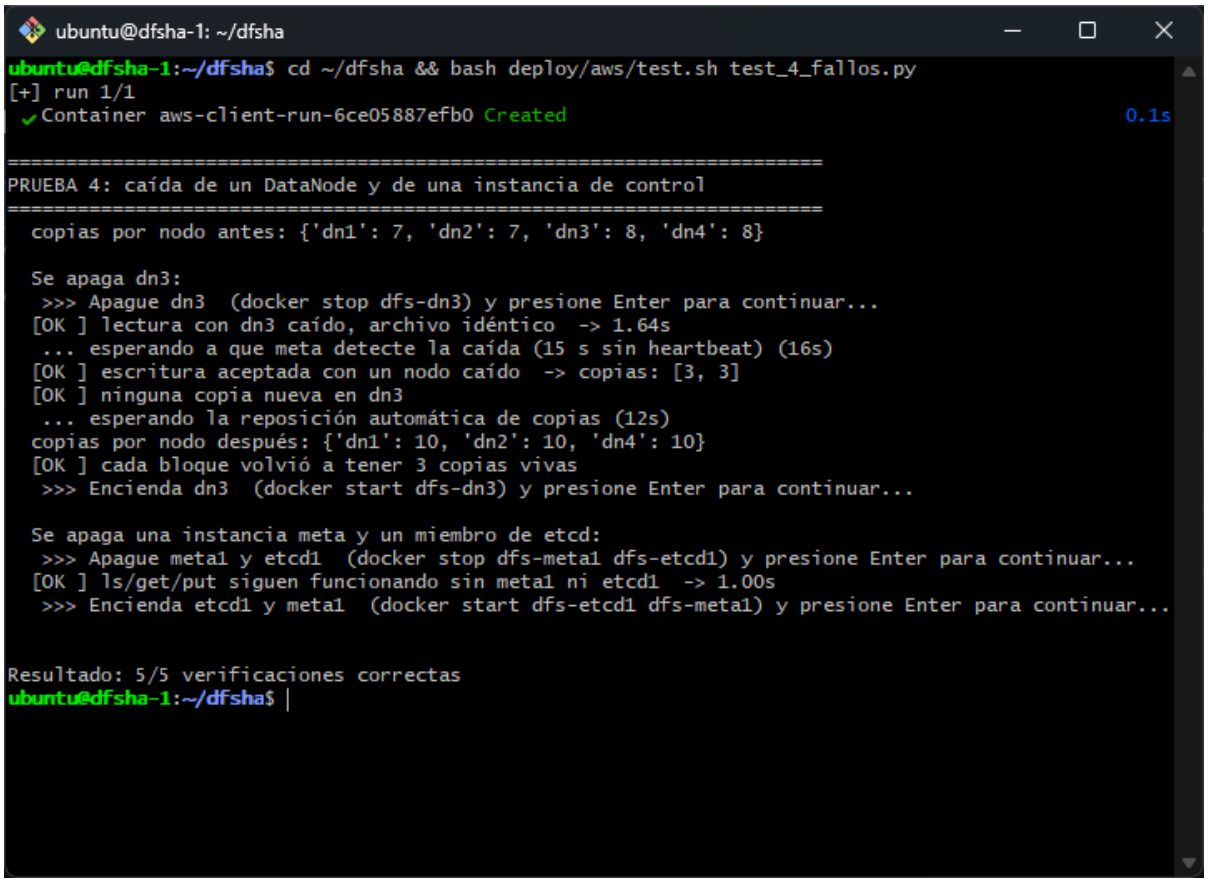
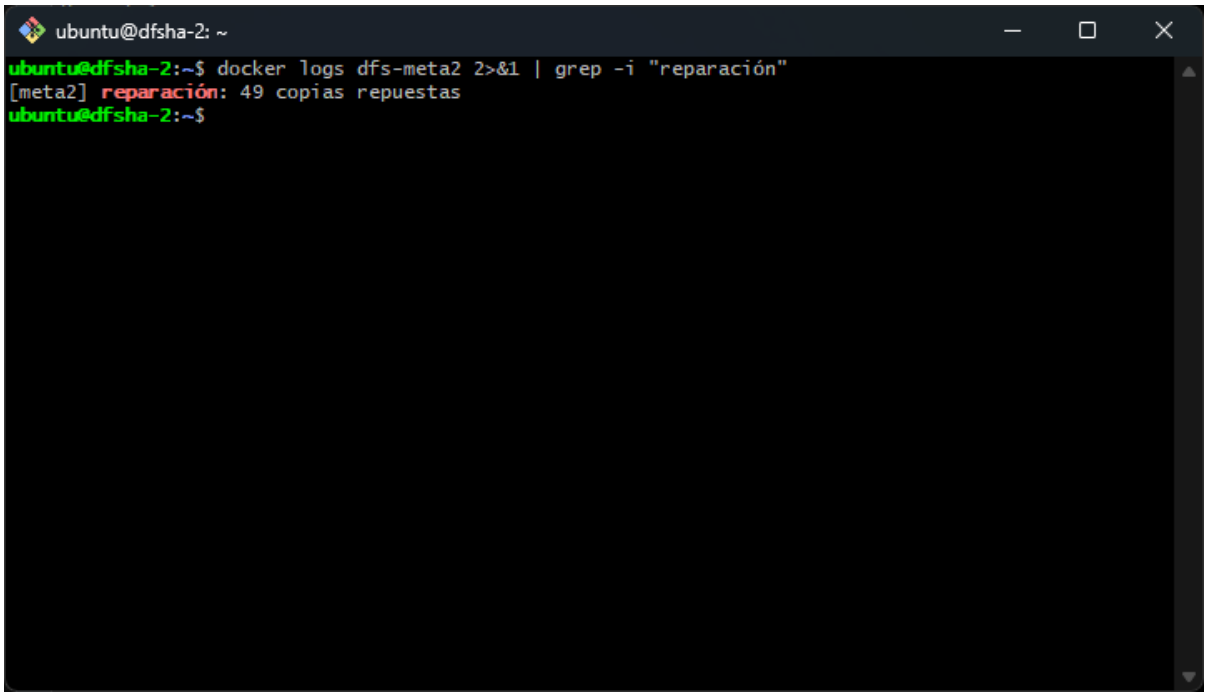


Figura 19

Registro de la reparación hecha por meta2



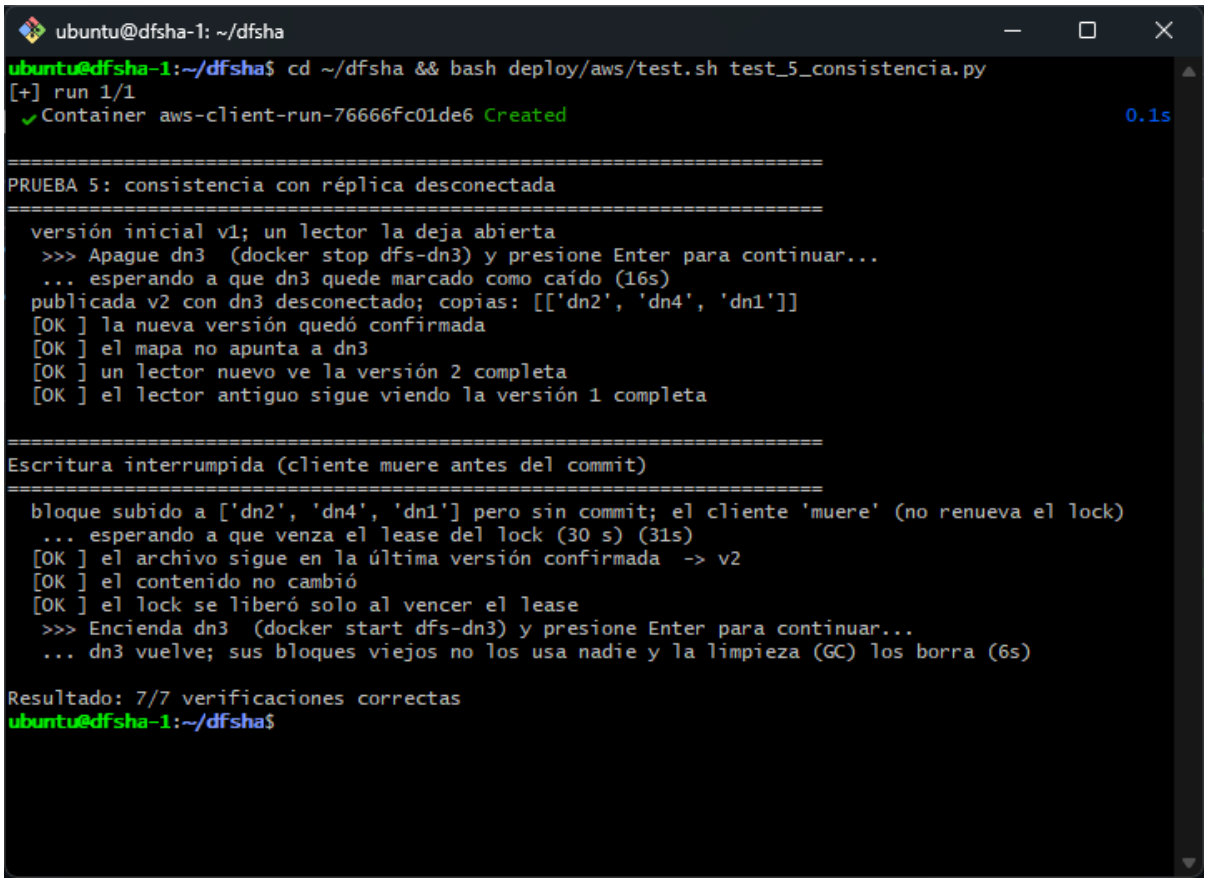
8.6 Prueba 5: consistencia

Con un lector abierto sobre la versión 1, se apagó dn3 y se publicó la versión 2. Sus copias quedaron en dn2, dn4 y dn1, y el mapa no apuntó a dn3. Un lector nuevo vio la versión 2 completa y el antiguo siguió viendo la 1 completa.

Después se simuló un cliente que sube un bloque a tres nodos y muere antes del commit. A los 30 s el lease de su lock venció, el lock se liberó solo y el archivo siguió en la versión 2 con su contenido intacto. El bloque huérfano queda para el GC.

Figura 20

Prueba 5: réplica desconectada y escritura interrumpida



8.7 Prueba 6: seguridad

Las 12 verificaciones pasaron (Tabla 8). La contraseña mala, la ausencia de token y el token falso dieron 401. El usuario abel no pudo leer los archivos de miguel ni escapar con .. (404). Los DataNodes rechazaron peticiones sin token, con token alterado, con el token de otro bloque y con un token de lectura usado para escribir, y meta no aceptó HTTP sin cifrar.

En disco, el bloque del archivo de prueba en dfs-dn2 ocupa 1028 bytes: los 1000 del texto más 12 del nonce y 16 de la etiqueta de AES-GCM. Buscar el texto original con grep devolvió 0 coincidencias, y od -c muestra bytes ilegibles. Por último, curl por HTTP terminó con el código 52 (sin respuesta), mientras que por HTTPS con la CA respondió {"meta":"meta1","etcd":"ok"}.

Figura 21

Prueba 6: autenticación, autorización y tokens

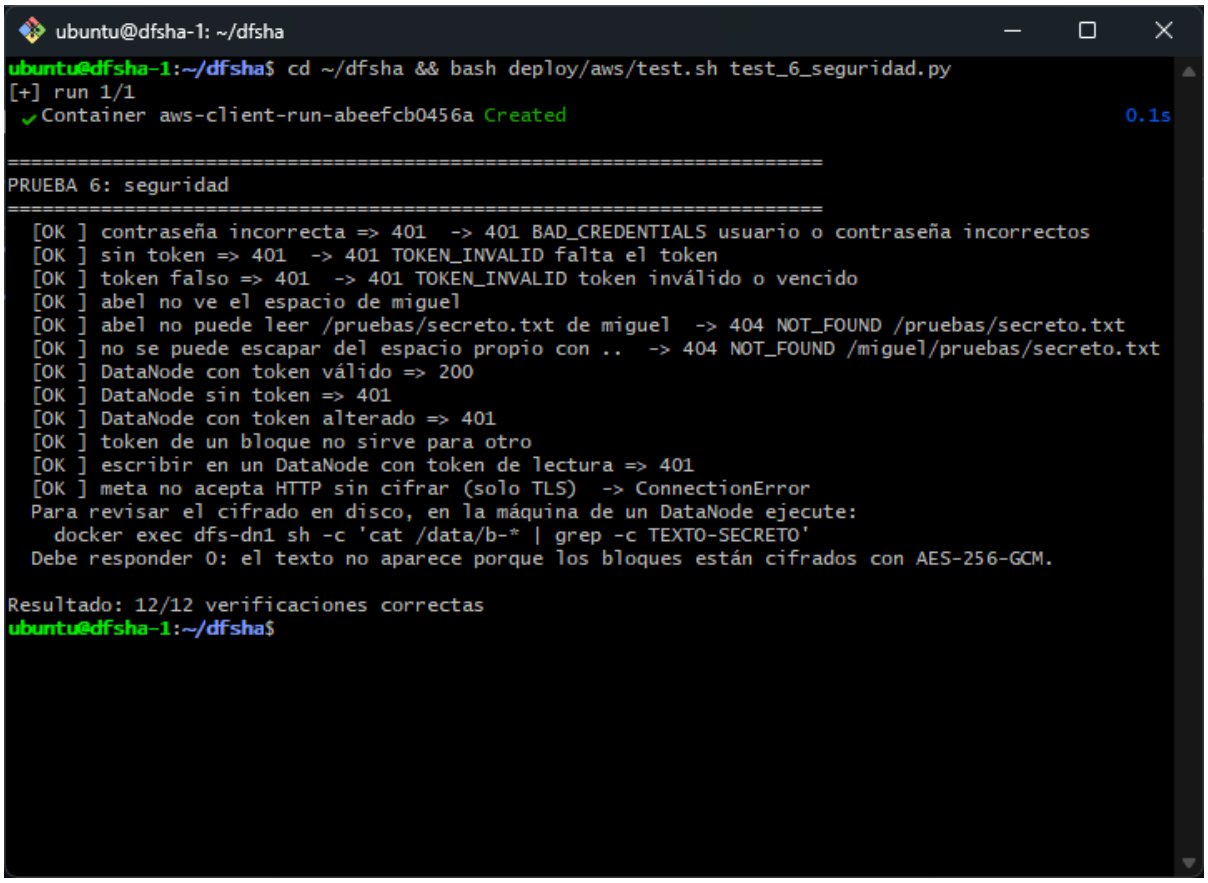


Figura 22

Cifrado en disco en dfs-dn2

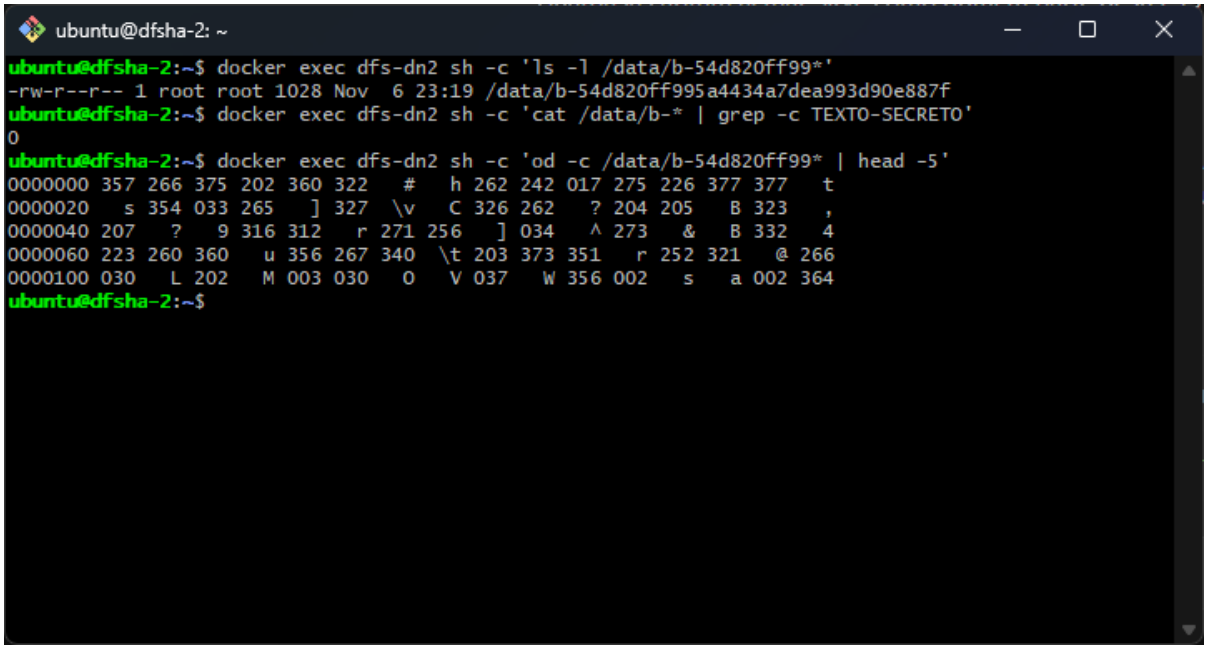
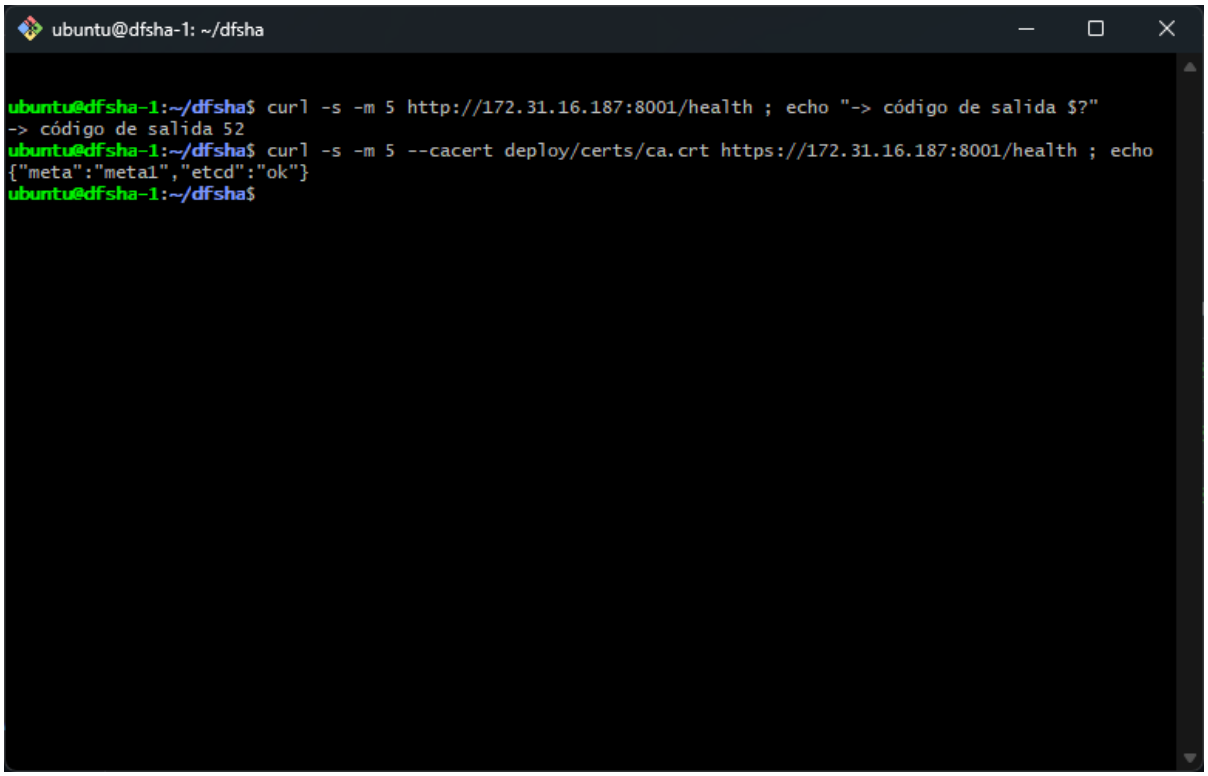


Figura 23

HTTP plano rechazado y HTTPS aceptado

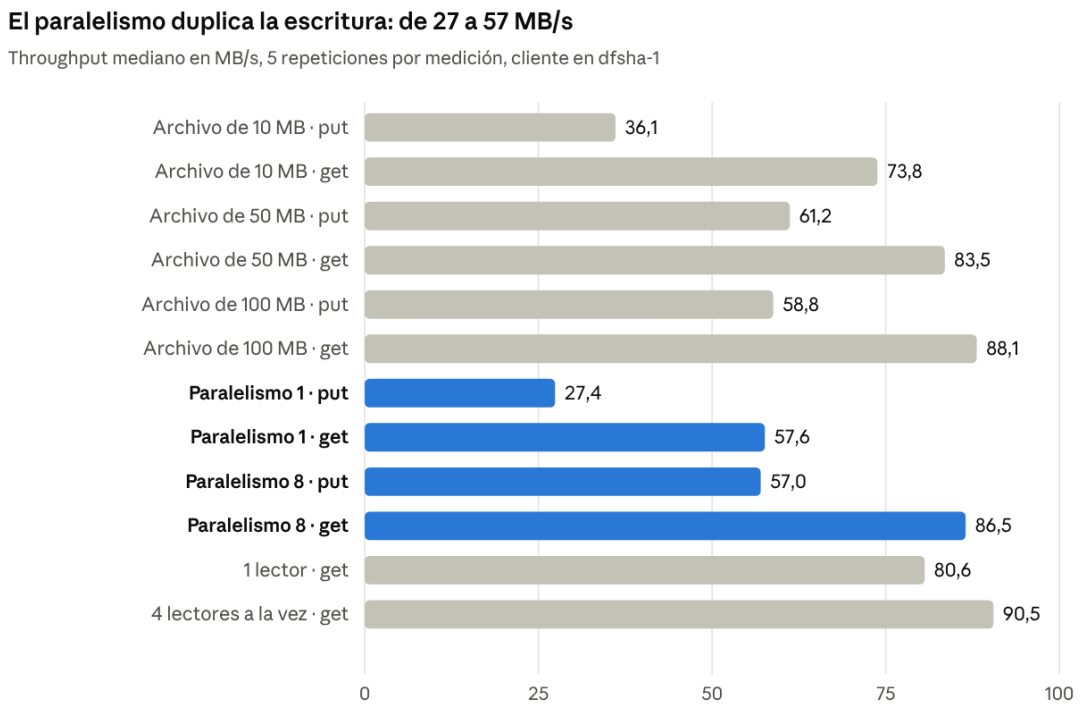


9. Rendimiento

Subir los bloques en paralelo fue la decisión que más pesó: con 8 bloques a la vez, la escritura de 100 MB pasó de 27,4 a 57,0 MB/s, y la lectura, de 57,6 a 86,5 MB/s.

Figura 24

Throughput mediano por condición (prueba 7, 5 repeticiones por medición)



Las lecturas superan a las escrituras en todas las condiciones (Figura 24), porque una lectura baja cada bloque de una sola réplica y una escritura lo sube a tres. El archivo de 10 MB es la excepción en escritura (36,1 MB/s): con solo 3 bloques pesan más el lock, la asignación y el commit que la transferencia. Con cuatro lectores simultáneos el throughput agregado subió a 90,5 MB/s, apenas por encima de un lector, lo que indica que el límite ya está en la red del cliente y no en los DataNodes.

Tabla 11

Latencias de la prueba 7

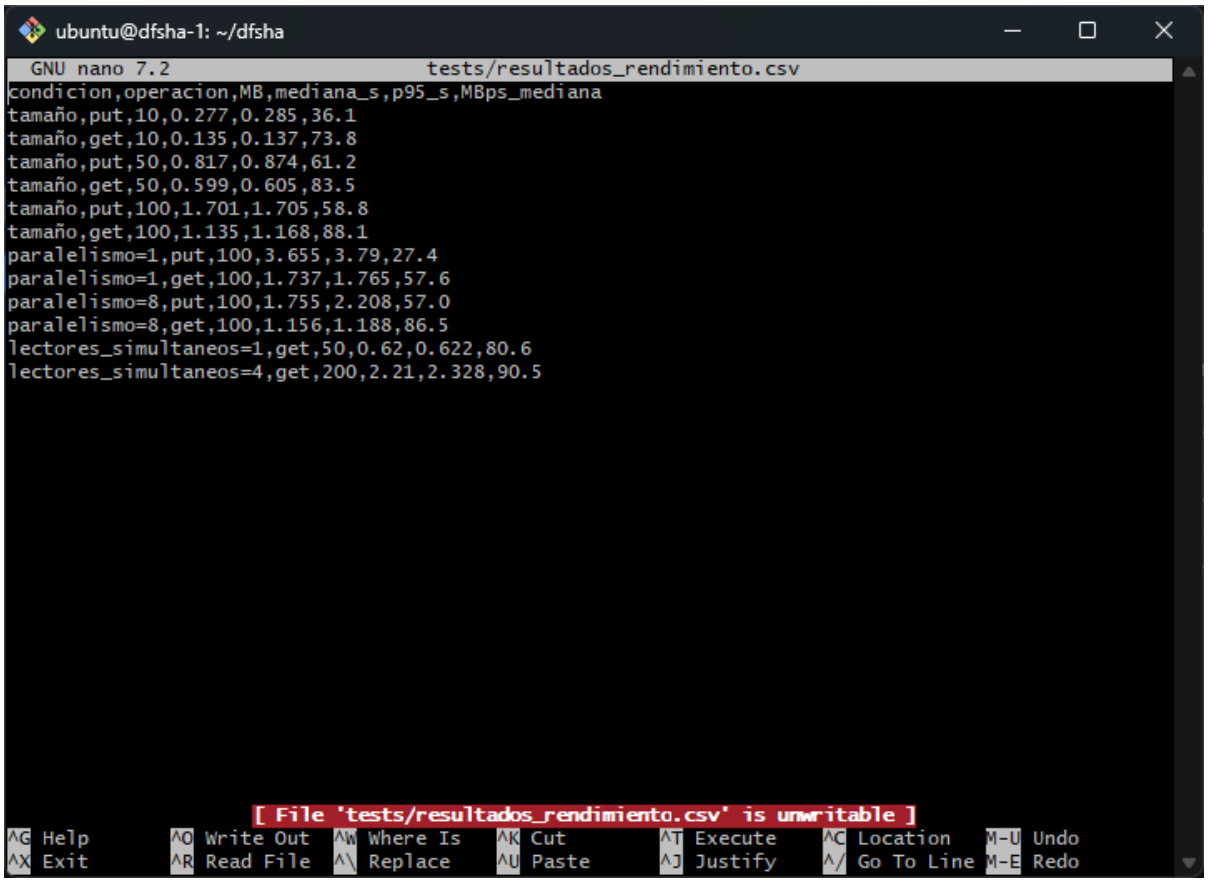
Medición	Mediana (s)	p95 (s)
put 10 MB	0,277	0,285
get 10 MB	0,135	0,137
put 50 MB	0,817	0,874
get 50 MB	0,599	0,605
put 100 MB	1,701	1,705
get 100 MB	1,135	1,168
put 100 MB, paralelismo 1	3,655	3,790
put 100 MB, paralelismo 8	1,755	2,208
get 100 MB, paralelismo 1	1,737	1,765
get 100 MB, paralelismo 8	1,156	1,188
get 50 MB, 1 lector	0,620	0,622

Medición	Mediana (s)	p95 (s)
get 4 × 50 MB, 4 lectores	2,210	2,328

La diferencia entre mediana y p95 no pasa del 7 % en 11 de las 12 mediciones, así que los tiempos fueron estables. La excepción es la escritura con paralelismo 8 (26 %), donde la carga simultánea sobre los cuatro nodos introduce más variación.

Figura 25

Resultados crudos de la prueba 7 (tests/resultados_rendimiento.csv)



10. Escenarios obligatorios, simplificaciones y limitaciones

10.1 Escenarios obligatorios

Tabla 12

Respuesta del sistema a los escenarios obligatorios

Escenario	Qué sucede	Evidencia
Dos procesos escriben el mismo bloque a la vez	El primero obtiene el lock; el segundo recibe 423, reintenta y escribe sobre la versión nueva. No se pierde ningún cambio	Prueba 3

Escenario	Qué sucede	Evidencia
Una réplica está desconectada durante una escritura	El bloque se sube a los nodos que responden; con W = 2 el commit se acepta y el mapa solo apunta a nodos vivos. La líder repone la tercera copia	Pruebas 4 y 5
Un nodo falla con un bloque a medias	El pedazo queda en <id>.part, que nunca se renombra y se borra al reiniciar. El commit exige W copias completas, así que nunca se publica un archivo incompleto	Prueba 5
El cliente no sabe dónde están los bloques	Solo conoce la lista de meta; GET /files/locate le da, por bloque, las direcciones de las réplicas vivas (RNF7)	Pruebas 1 y 4

10.2 Simplificaciones frente a los diseños individuales

Las simplificaciones respondieron a un criterio: quitar lo que cuesta mucho programar sin debilitar ninguna garantía declarada.

1. Sin gRPC: todo el tráfico es REST sobre HTTPS.
2. Sin replicación en cadena: el cliente escribe en paralelo a las réplicas.
3. Lock por archivo, no por rango de bloques.
4. mTLS solo en etcd; entre meta y los DataNodes se usa HTTPS con un token interno compartido.
5. Primitivas de RF3: open(r|w), read, write, lock, unlock y close. Quedan fuera mv, flush y los modos a y r+, porque write(offset) ya cubre el anexado y la modificación parcial.
6. Sin versiones históricas: el GC borra las versiones viejas tras 10 minutos.
7. Una sola llave de cifrado en reposo por despliegue, sin derivar una por bloque.

10.3 Limitaciones conocidas

- **Detección de caídas en unos 15 s.** Durante ese tiempo, alloc puede proponer el nodo caído; el cliente lo salta y prueba el siguiente, pero la escritura tarda algo más.
- **Quórum de etcd.** Con 2 de 3 miembros caídos no hay operaciones de metadatos: el sistema prefiere detenerse a divergir.
- **Dos DataNodes en una misma máquina.** dn3 y dn4 comparten dfsha-3, así que la caída de esa EC2 puede llevarse dos de las tres copias de un bloque. Con una cuarta EC2 se resolvería.
- **Escritores de bloques distintos se serializan,** porque el lock es por archivo.
- **Sin rebalanceo.** Un DataNode nuevo recibe bloques nuevos, pero no se le mueven los existentes.

11. Conclusiones

Al empezar la fase grupal parecía que la alta disponibilidad dependía de cuánto código de coordinación fuéramos capaces de escribir. Las pruebas mostraron lo contrario. El sistema sobrevivió a la caída simultánea de un servidor de control y un miembro de etcd sin una sola línea de consenso propio, porque esa responsabilidad se delegó a una herramienta diseñada para ella.

La decisión más importante fue reducir el número de lugares donde el sistema puede quedar inconsistente. Al hacer los bloques inmutables y concentrar cada cambio en una sola transacción, los fallos dejaron de ser casos especiales: un bloque a medias no se ve, un commit interrumpido no publica nada y un escritor pausado no puede imponer datos viejos. La prueba 5 lo evidenció cuando un cliente murió con un bloque ya subido y el archivo no cambió.

En el rendimiento también hubo un equilibrio. Subir cada bloque a tres nodos triplica el trabajo de red del cliente frente a la copia en cadena; a cambio, el código es más simple y un nodo caído se esquiva sin reorganizar nada. Con 8 bloques en paralelo, las escrituras alcanzaron entre 57 y 61 MB/s y las lecturas entre 83 y 90 MB/s, cifras suficientes para el alcance del curso.

Queda abierta una pregunta para una versión futura: hasta dónde se puede crecer antes de que el grupo Raft de etcd se vuelva el techo. Repartir los usuarios entre varios clústeres, agregar rebalanceo y locks por rango de bloques, y separar cada DataNode en su propia máquina son el siguiente paso natural. La base, sin embargo, ya demuestra lo esencial: un sistema distribuido confiable no depende tanto de cuánto se construye como de saber qué no construir.

Referencias

Burrows, M. (2006). The Chubby lock service for loosely-coupled distributed systems. *Proceedings of OSDI*.

etcd Authors. (2024). *etcd v3.5 documentation*. https://etcd.io/docs/v3.5/

Ghemawat, S., Gobioff, H., y Leung, S.-T. (2003). The Google File System. *Proceedings of SOSP*.

Gray, C., y Cheriton, D. (1989). Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. *Proceedings of SOSP*.

Howard, J. H., et al. (1988). Scale and performance in a distributed file system. *ACM Transactions on Computer Systems, 6*(1).

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly.

Ongaro, D., y Ousterhout, J. (2014). In search of an understandable consensus algorithm. *Proceedings of USENIX ATC*.

Shvachko, K., Kuang, H., Radia, S., y Chansler, R. (2010). The Hadoop Distributed File System. *Proceedings of IEEE MSST*.

van Steen, M., y Tanenbaum, A. S. (2023). *Distributed systems* (4.ª ed.).